Array Manipulation

Find the second-highest number in an array.

Simple Explanation: The simplest way is to sort the entire array in descending order first. The element at the second position (index 1) will be the second-highest number. Alternatively, you can track the highest and second-highest numbers in a single pass through the array without sorting.

```
public class SecondHighestFinder {
    public static int findSecondHighest(int[] arr) {
        if (arr == null || arr.length < 2) {
            throw new IllegalArgumentException("Array must have at least
two elements.");
        }

        int max = Integer.MIN_VALUE;
        int secondMax = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > max) {
                // New maximum found, update secondMax to old max
                secondMax = max;
                max = num;
            } else if (num > secondMax && num != max) {
                // Found a number greater than secondMax but not equal to
max
                secondMax = num;
            }
        }

        // Handle case where all elements are the same (e.g., {5, 5, 5})
        if (secondMax == Integer.MIN_VALUE) {
            return max; // Or throw error, depending on requirement
        }

        return secondMax;
    }
}
```

Move all zeros to the end of an array.

Simple Explanation: We use a pointer, usually starting at index 0, to track where the next non-zero element should be placed. We loop through the array; if we find a **non-zero** number, we move it to the current pointer position and then move the pointer one step forward. After the loop, we fill the remaining positions from the pointer onward with zeros.

```
public class ZeroMover {
    public static void moveZeros(int[] arr) {
        if (arr == null || arr.length == 0) {
            return;
        }

        int nonZeroPointer = 0; // Tracks the position for the next non-
zero element

        // Phase 1: Move all non-zero elements to the front
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] != 0) {
                arr[nonZeroPointer] = arr[i];
                nonZeroPointer++;
            }
        }

        // Phase 2: Fill the rest of the array with zeros
        for (int i = nonZeroPointer; i < arr.length; i++) {
            arr[i] = 0;
        }
    }
}
```


Check if the array is sorted.

Simple Explanation: To check if an array is sorted (either ascending or descending), we loop through the array starting from the second element. In each step, we compare the current element with the element immediately before it. If we find any element that is smaller than its previous element (for ascending), the array is not sorted.

```
public class SortedChecker {
    public static boolean isSortedAscending(int[] arr) {
        if (arr == null || arr.length <= 1) {
            return true;
        }

        // Check for ascending order
        for (int i = 1; i < arr.length; i++) {
            // If the current element is smaller than the previous one,
it's not sorted
```

```
            if (arr[i] < arr[i - 1]) {
                return false;
            }
        }
        return true;
    }
}
```

Find the smallest and largest element in an array.

Simple Explanation: We initialize two variables, min and max, with the value of the first element in the array. Then, we loop through the rest of the array, comparing each number to our current min and max values. If a number is smaller than min, we update min. If it's larger than max, we update max.

```
public class MinMaxFinder {
    public static void findMinMax(int[] arr) {
        if (arr == null || arr.length == 0) {
            System.out.println("Array is empty.");
            return;
        }

        // Initialize min and max with the first element
        int min = arr[0];
        int max = arr[0];

        // Start loop from the second element
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
            if (arr[i] > max) {
                max = arr[i];
            }
        }

        System.out.println("Smallest: " + min);
        System.out.println("Largest: " + max);
    }
}
```

Left rotate array by one position.

Simple Explanation: Left rotation by one position means the first element moves to the very end of the array, and all other elements shift one position to the left. We save the first element temporarily, shift all elements from index 1 up to the end, and finally put the saved first element into the last position.

```
public class ArrayRotator {
    public static void leftRotateByOne(int[] arr) {
        if (arr == null || arr.length <= 1) {
            return;
        }

        // 1. Save the first element
        int firstElement = arr[0];

        // 2. Shift all other elements one position to the left
        for (int i = 0; i < arr.length - 1; i++) {
            arr[i] = arr[i + 1];
        }

        // 3. Put the saved first element into the last position
        arr[arr.length - 1] = firstElement;
    }
}
```